Final Project: LU Factorization

Nicholas Storti

August 10, 2026

1 Introduction

A common problem in electrical engineering is solving systems of linear equations, which can be represented in the form $\mathbf{Ax} = \mathbf{b}$. In this expression, $\mathbf{A}$ is the matrix of coefficients, $\mathbf{b}$ is a known vector, and $\mathbf{x}$ is the solution vector to be solved for. Systems of linear equations frequently show up in electric and magnetic circuit problems, often involving systems of nodal and loop equations. The solutions to these equations provide values for voltages at nodes and currents through each path. One technique for solving the linear system $\mathbf{Ax} = \mathbf{b}$ is to find the inverse of matrix $\mathbf{A}$ and use the inverse to find $\mathbf{x}$ by:

$$\mathbf{A^{-1}Ax} = \mathbf{A^{-1}b}$$

$$\mathbf{Ix} = \mathbf{A^{-1}b}$$

$$\mathbf{x} = \mathbf{A^{-1}b}$$

Computing $\mathbf{A^{-1}}$ explicitly, however, is computationally intensive and unnecessary for solving the linear system. A better method is to compute the vector $\mathbf{A^{-1}b}$ directly, and some techniques for accomplishing this include LU and QR factorization of matrix $\mathbf{A}$.

The topic of this report is a parallel implementation of an LU factorization algorithm. Specifically, the right-looking Doolittle algorithm is implemented in Cuda C to perform parallel LU factorization of matrix $\mathbf{A}$ without pivoting. The report is organized as follows: 2) background section detailing how LU factorization can be used to solve the linear system $\mathbf{Ax} = \mathbf{b}$ and explanation of the right-looking LU factorization algorithm, 3) implementation section detailing three different parallel implementations of the LU factorization algorithm, 4) output section with the output times for different input matrix sizes for each implementation, 5) analysis section where the performance of each implementation is discussed, and 6) conclusion.

2 Background

2.1 Linear System Solution Using LU Factorization

LU factorization is the process of factoring a matrix $\mathbf{A}$ into a lower triangular matrix, $\mathbf{L}$, and an upper triangular matrix, $\mathbf{U}$. For 3x3 $\mathbf{A}$ this looks like:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} = \mathbf{LU}$$

Substituting $\mathbf{A} = \mathbf{LU}$ into $\mathbf{Ax} = \mathbf{b}$ yields [1]:

$$\mathbf{Ax} = \mathbf{b}$$

$$\mathbf{LUx} = \mathbf{b}$$

$$\mathbf{Ux} = \mathbf{L^{-1}b}$$

$$\mathbf{x} = \mathbf{U^{-1}}(\mathbf{L^{-1}b})$$

If we let $\mathbf{y} = \mathbf{Ux}$, $\mathbf{Ax} = \mathbf{b}$ becomes [1]:

$$\mathbf{Ax} = \mathbf{b}$$
$$\mathbf{LUx} = \mathbf{b}$$
$$\mathbf{Ly} = \mathbf{b}$$
$$\mathbf{Ux} = \mathbf{y}$$

We can then solve for $\mathbf{x}$ by first solving the linear system $\mathbf{Ly} = \mathbf{b}$ for $\mathbf{y}$, then solving the linear system $\mathbf{Ux} = \mathbf{y}$ for $\mathbf{x}$. $\mathbf{Ly} = \mathbf{b}$ can be solved relatively easily using forward substitution since $\mathbf{L}$ is lower triangular. Similarly, $\mathbf{Ux} = \mathbf{y}$ can be solved easily using backward substitution since $\mathbf{U}$ is upper triangular.

This is how the LU factorization of $\mathbf{A}$ can be used to solve the linear system $\mathbf{Ax} = \mathbf{b}$. While the explanation for how the LU factorization can be used is presented, the primary focus of this report is computation of the LU factorization itself on a parallel processor (GPU). The remainder of the report will deal with parallel implementation of an LU factorization algorithm.

2.2 Right-Looking Doolittle LU Factorization Algorithm

The right-looking algorithm is an alternative to Gaussian elimination for calculating $\mathbf{L}$ and $\mathbf{U}$. First, $\mathbf{A} = \mathbf{LU}$ can be expressed as [1]:

$$\begin{bmatrix} a_{11} & \boldsymbol{a_{12}} \\ \boldsymbol{a_{21}} & \boldsymbol{A_{22}} \end{bmatrix} = \begin{bmatrix} 1 & \mathbf{0} \\ \boldsymbol{l_{21}} & \boldsymbol{L_{22}} \end{bmatrix} \begin{bmatrix} u_{11} & \boldsymbol{u_{12}} \\ \mathbf{0} & \boldsymbol{U_{22}} \end{bmatrix} = \begin{bmatrix} u_{11} & \boldsymbol{u_{12}} \\ u_{11}\boldsymbol{l_{21}} & (\boldsymbol{l_{21}u_{12}} + \boldsymbol{L_{22}U_{22}}) \end{bmatrix}$$

If $\mathbf{A}$ is $n \times n$, then u_{11} is a scalar diagonal element of $\mathbf{U}$, $\boldsymbol{u_{12}}$ is a $1 \times n$ row vector of $\mathbf{U}$, $\boldsymbol{l_{21}}$ is a $n \times 1$ column vector of $\mathbf{L}$, and $\boldsymbol{L_{22}}$, $\boldsymbol{U_{22}}$ are $(n-1) \times (n-1)$ trailing submatrices of $\mathbf{L}$ and $\mathbf{U}$. The elements of $\mathbf{L}$ and $\mathbf{U}$ can then be solved for by [1]:

$$u_{11} = a_{11}$$
$$\boldsymbol{u_{12}} = \boldsymbol{a_{12}}$$
$$\boldsymbol{l_{21}} = \frac{1}{u_{11}}\boldsymbol{a_{21}}$$
$$\boldsymbol{L_{22}U_{22}} = \boldsymbol{A_{22}} - \boldsymbol{l_{21}u_{12}}$$

$\boldsymbol{L_{22}U_{22}}$ is the Schur complement $\boldsymbol{S_{22}}$ of $\boldsymbol{A_{22}}$. The previous calculations can then recursively be used on $\boldsymbol{S_{22}}$ to find the entirety of $\mathbf{L}$ and $\mathbf{U}$ for $\mathbf{A}$. The steps of the algorithm are:

1. Update current row of $\mathbf{U}$ with current row of $\mathbf{A}$ by $\boldsymbol{u_{12}} = \boldsymbol{a_{12}}$.
2. Calculate current column of $\mathbf{L}$ using $\boldsymbol{l_{21}} = \frac{1}{u_{11}}\boldsymbol{a_{21}}$.
3. Calculate Schur complement of trailing submatrix using $\boldsymbol{A_{22}} = \boldsymbol{A_{22}} - \boldsymbol{l_{21}u_{12}}$. Elements of trailing submatrix $\boldsymbol{A_{22}}$ are overwritten in $\mathbf{A}$ with the Schur complement so that step 1 will be valid for the next iteration.
4. Recursively go back to step 1 on trailing submatrix $\boldsymbol{A_{22}}$ until all of $\mathbf{L}$ and $\mathbf{U}$ are filled out.

3 Implementation

3.1 Naive Approach

Since each iteration of the algorithm depends on the previous iteration, this aspect must be performed serially and cannot be parallelized. On each iteration, however, there are aspects that can be parallelized. Specifically, the $\mathbf{L}$ column update in step 2 and the Schur complement trailing submatrix update in step 3 on each iteration can be parallelized. Two separate kernels are implemented for each of these calculations.

The naive implementation assigns a single thread to each output element.

```
__global__ void luColUpdateKernel_V1(float *out, unsigned int size, unsigned int k) {
 int i = k + 1 + blockIdx.x*blockDim.x + threadIdx.x;

 if(i < size) {
  out[i*size + k] /= out[k*size + k];
 }
}

__global__ void luTrailingUpdateKernel_V1(float *out, unsigned int size, unsigned int k) {
 int i = k + 1 + blockIdx.y * blockDim.y + threadIdx.y;
 int j = k + 1 + blockIdx.x * blockDim.x + threadIdx.x;

 if(i < size && j < size) {
  out[i*size + j] -= out[i*size + k] * out[k*size + j];
 }
}
```

3.2 Block Cyclic Coarsening

This allows for better work efficiency and higher thread utilization.

3.3 Block Cyclic with Shared Memory Tiling

Tiling reduces global memory accesses.

4 Output

4.1 Naive Implementation

```
Setting up the problem...0.000011 s
Input size = 16
Allocating device variables...0.131343 s
Copying data from host to device...0.000040 s
Launching kernel V1...0.101835 s
Copying data from device to host...0.000023 s
Verifying results...TEST PASSED


Setting up the problem...0.000103 s
Input size = 64
Allocating device variables...0.117873 s
Copying data from host to device...0.000058 s
Launching kernel V1...0.101291 s
Copying data from device to host...0.000027 s
Verifying results...TEST PASSED


Setting up the problem...0.000856 s
Input size = 200
Allocating device variables...0.116584 s
Copying data from host to device...0.000092 s
Launching kernel V1...0.102618 s
Copying data from device to host...0.000176 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.005246 s
Input size = 500
Allocating device variables...0.117505 s
Copying data from host to device...0.000331 s
Launching kernel V1...0.105834 s
Copying data from device to host...0.000956 s
Verifying results...TEST PASSED


Setting up the problem...0.020418 s
Input size = 1000
Allocating device variables...0.116747 s
Copying data from host to device...0.001030 s
Launching kernel V1...0.113392 s
Copying data from device to host...0.002477 s
Verifying results...TEST PASSED


Setting up the problem...0.079548 s
Input size = 2000
Allocating device variables...0.118421 s
Copying data from host to device...0.003544 s
Launching kernel V1...0.148341 s
Copying data from device to host...0.006362 s
Verifying results...TEST PASSED


Setting up the problem...0.494852 s
Input size = 5000
Allocating device variables...0.118831 s
Copying data from host to device...0.021155 s
Launching kernel V1...0.599423 s
Copying data from device to host...0.035856 s
Verifying results...TEST PASSED
```

4.2 Block Cyclic

```
Setting up the problem...0.000013 s
Input size = 16
Allocating device variables...0.117475 s
Copying data from host to device...0.000046 s
Launching kernel V2...0.100952 s
Copying data from device to host...0.000024 s
Verifying results...TEST PASSED


Setting up the problem...0.000103 s
Input size = 64
Allocating device variables...0.117988 s
Copying data from host to device...0.000058 s
Launching kernel V2...0.100843 s
Copying data from device to host...0.000027 s
```

```
Verifying results...TEST PASSED


Setting up the problem...0.000849 s
Input size = 200
Allocating device variables...0.116893 s
Copying data from host to device...0.000090 s
Launching kernel V2...0.101611 s
Copying data from device to host...0.000179 s
Verifying results...TEST PASSED


Setting up the problem...0.005265 s
Input size = 500
Allocating device variables...0.118979 s
Copying data from host to device...0.000331 s
Launching kernel V2...0.104408 s
Copying data from device to host...0.000941 s
Verifying results...TEST PASSED


Setting up the problem...0.020434 s
Input size = 1000
Allocating device variables...0.116915 s
Copying data from host to device...0.001020 s
Launching kernel V2...0.116745 s
Copying data from device to host...0.002470 s
Verifying results...TEST PASSED


Setting up the problem...0.079565 s
Input size = 2000
Allocating device variables...0.118208 s
Copying data from host to device...0.003524 s
Launching kernel V2...0.208327 s
Copying data from device to host...0.006385 s
Verifying results...TEST PASSED


Setting up the problem...0.494968 s
Input size = 5000
Allocating device variables...0.118387 s
Copying data from host to device...0.021173 s
Launching kernel V2...2.040773 s
Copying data from device to host...0.035730 s
Verifying results...TEST PASSED
```

4.3 Block Cyclic with Shared Memory Tiling

```
Setting up the problem...0.000014 s
Input size = 16
Allocating device variables...0.117405 s
Copying data from host to device...0.000040 s
Launching kernel V3...0.101178 s
```

```
Copying data from device to host...0.000023 s
Verifying results...TEST PASSED


Setting up the problem...0.000103 s
Input size = 64
Allocating device variables...0.117837 s
Copying data from host to device...0.000058 s
Launching kernel V3...0.100756 s
Copying data from device to host...0.000028 s
Verifying results...TEST PASSED


Setting up the problem...0.000855 s
Input size = 200
Allocating device variables...0.116941 s
Copying data from host to device...0.000093 s
Launching kernel V3...0.101196 s
Copying data from device to host...0.000185 s
Verifying results...TEST PASSED


Setting up the problem...0.005244 s
Input size = 500
Allocating device variables...0.117837 s
Copying data from host to device...0.000331 s
Launching kernel V3...0.103023 s
Copying data from device to host...0.000940 s
Verifying results...TEST PASSED


Setting up the problem...0.020438 s
Input size = 1000
Allocating device variables...0.117181 s
Copying data from host to device...0.001030 s
Launching kernel V3...0.111351 s
Copying data from device to host...0.002467 s
Verifying results...TEST PASSED


Setting up the problem...0.079559 s
Input size = 2000
Allocating device variables...0.118131 s
Copying data from host to device...0.003535 s
Launching kernel V3...0.168336 s
Copying data from device to host...0.006392 s
Verifying results...TEST PASSED


Setting up the problem...0.494934 s
Input size = 5000
Allocating device variables...0.118447 s
Copying data from host to device...0.021156 s
Launching kernel V3...0.948372 s
```

```
Copying data from device to host...0.035792 s
Verifying results...TEST PASSED
```

5 Analysis

Naive implementation was the worst.

6 Conclusion

This was a decent project.

7 References

1. https://courses.grainger.illinois.edu/cs357/sp2020/notes/ref-9-linsys.html